

Proyecto Final: Tema deportivo enmarcado en un universo especial.

Deporte Kendo en el universo de Kill Bill

Isabella Carvajal Cuartas

C.C: 1094903251

Informática II

Universidad de Antioquia

2026-1

Tabla de contenido

- 1. Introducción**
- 2. Planteamiento del problema**
- 3. Definición general y objetivos**
- 4. Especificación de requerimientos**
- 5. Diseño y arquitectura del sistema**
- 6. Dependencias externas y bibliotecas**
- 7. Desarrollo e implementación**
- 8. Procedimientos de prueba**
- 9. Controles del juego**
- 10. Resultados y discusión**
- 11. Conclusiones**
- 12. Referencias**

1.Introducción

Este documento describe el diseño, la implementación y el funcionamiento de un videojuego de combate 2D/2.5D desarrollado en C++ utilizando el framework Qt. El proyecto toma como inspiración temática la película Kill Bill de Quentin Tarantino, recreando enfrentamientos de esgrima japonesa (kendo) entre personajes icónicos de la saga.

El sistema fue concebido como un ejercicio académico de programación orientada a objetos avanzada, enfocándose en la correcta aplicación de herencia, patrones de señales/slots, manejo de física de movimiento y diseño de inteligencia artificial para personajes no jugables.

2. Planteamiento del Problema

Los proyectos académicos de programación orientada a objetos frecuentemente se limitan a ejercicios abstractos que no permiten visualizar la aplicabilidad real de los conceptos. La ausencia de proyectos integradores que combinen física de movimiento, animación por sprites, inteligencia artificial adaptativa y manejo de estados en tiempo real dificulta la comprensión profunda de estos paradigmas.

Este proyecto responde a esa necesidad desarrollando un videojuego funcional donde cada componente tiene una justificación técnica directa en los conceptos de POO: desde la física de salto parabólico hasta el algoritmo de ajuste de probabilidades del enemigo final. Para permitir englobar los conocimientos aprendidos durante el curso.

3. Definición General y Objetivos

3.1 Descripción general

De acuerdo con el deporte de Kendo al no ser muy conocido en la cultura colombiana, se harán ciertas especificaciones de las palabras a utilizar referentes a este. Hay diferentes zonas válidas para este deporte en la vida real. Sin embargo, por simplicidad se reducirán a tres: alto (men), medio (do) y bajo (kote). Los ataques encadenados se denominan renzoku-waza y un ataque válido precedido de otro se denomina zanshin.

El juego cuenta con dos niveles de combate distintos. En el Nivel 1, la jugadora (The Bride) se enfrenta a O-Ren Ishii en un duelo de kendo 2D lateral clásico, con sistema de puntos (5 golpes para ganar) y mecánica especial de Zanshin. En el Nivel 2, se enfrenta a Bill en un entorno 2.5D, sistema de parry y dificultad adaptativa de acuerdo al personaje de Bill.

3.2 Objetivos técnicos

- Implementar jerarquía de clases con herencia y polimorfismo

- Aplicar física de movimiento: rectilíneo, parabólico, así como movimiento acelerado y desacelerado.
- Desarrollar IA con comportamiento adaptativo basado en estadísticas del oponente
- Gestionar estados de juego y transiciones de escena
- Integrar audio y animación por spritesheet automatizada

3.3 Objetivos académicos

- Demostrar dominio de POO en un contexto real y funcional
- Gestionar memoria dinámica de forma correcta y sin fugas
- Implementar algoritmos de detección de colisión por hitbox
- Separar responsabilidades en clases cohesivas y bien definidas

4. Especificación de Requerimientos

4.1 Requisitos funcionales

- El juego debe presentar un menú principal con opciones para iniciar cada nivel directamente y salir.
- El Nivel 1 debe implementar un sistema de puntos (máximo 5 puntos gana) y mecánica de Zanshin acumulable.
- El Nivel 2 debe ofrecer dos modos de dificultad: Easy y hardcore, representadas con aprendiz y maestro para bill con comportamiento de IA diferenciado.
- La jugadora debe poder realizar ataques a tres zonas: Men (cabeza), Do (torso) y Kote (pies).
- Al ganar o perder, el juego debe mostrar pantalla de resultado con opciones de reinicio o menú.
- Debe reproducirse música de fondo diferente en menú y cada nivel, con efectos de sonido para golpes y gritos.

4.2 Requisitos no funcionales

- El ciclo de juego debe actualizarse con timer de 16 ms.
- La resolución de escena fija es 1200x630 pixeles.
- Los sprites deben cargarse desde recursos embebidos (.qrc), sin dependencias de rutas externas ya que es fundamental que se encuentren allí para cargarlos.

- La IA de Bill debe ajustar probabilidades de ataque con mínimo 1-2 interacciones de aprendizaje.

5. Diseño y Arquitectura del Sistema

5.1 Estructura de capas

El sistema se organiza en cuatro capas con responsabilidades delimitadas:

- Capa de interfaz gráfica: QGraphicsView, múltiples QGraphicsScene (menú, juego, carga, derrota, victoria, dificultad) y botones QPushButton con fondo transparente superpuestos sobre los fondos de imagen, ya que se decidió personalizar los fondos en las cargas entre niveles, así mismo como las imágenes para victoria y derrota.
- Capa de control: La clase Juego actúa como cerebro central gestionando el QTimer del ciclo de juego, la música, los estados del juego (EstadoJuego) y las transiciones. La clase Nivel gestiona cada nivel individualmente, apropiándose de los requerimientos impuestos en cada nivel y las limitaciones por personaje. Así mismo maneja los requisitos del usuario por teclado.
- Capa de dominio/personajes: jerarquía Personaje (abstracta) la cual Bride, Oren, Bill heredan de ella. Cada personaje encapsula su física, animación, IA y lógica de combate.
- Capa de recursos: Sprites (QPixmap, hojas 128x128 por frame) la cual para todos los personajes tiene ciertas características por filas dependiendo del estado, dirección y tipo de ataque, audio (QMediaPlayer para musica, QSoundEffect para efectos) y fondos, todos dentro del archivo .qrc.

5.2 Resumen de clases (El diagrama completo se encuentra con el archivo de *diagramaClases.pdf* así mismo como su versión resumen *diagramaClasesResumen.pdf*)

Clase	Hereda de	Responsabilidad principal	Señales clave
Juego	QObject	Orquestador central. Gestiona escenas, música, estados de juego y transiciones entre niveles.	juegoTerminadoVictoria, juegoTerminadoDerrota
Nivel	QObject	Gestiona un nivel concreto: carga personajes, detecta	victoria(), derrota()

Clase	Hereda de	Responsabilidad principal	Señales clave
		colisiones, procesa input y actualiza marcador.	
Personaje	QGraphicsObject	Clase base abstracta. Define física, Sprite, hitboxes, estados y zonas de ataque.	—
Bride	Personaje	Personaje jugable. Implementa modos de física, sistema Zanshin y mecánica de Parry.	zanshinEspecialIniciado, zanshinEspecialterminado, zanshinActualizado
Oren	Personaje	IA del nivel 1. Decide acercarse, atacar o retroceder según distancia. Modo debilitado.	—
Bill	Personaje	IA adaptativa del nivel 2. Aprende de ataques y parrys y ajusta probabilidades de ataque.	entroEnVulnerabilidad()

5.3 Enumeraciones clave

De acuerdo con lo investigado en [1] se logró llegar a una solución más compacta para diferentes eventos que ocurren durante el juego, haciendo uso de enumeraciones:

- Estado: Quieto, Moviose, Atacando, Defendiendo, Saltando, Vulnerable, Golpeado, Derrotado
- ZonaAtaque: Men, Do, Kote, Inicial
- Direccion: Adelante, Atras, Derecha, Izquierda
- EstadoJuego: Menu, Cargando, Nivel1, Nivel2, Victoria, Derrota
- ModoFisica: Uniforme, Parabolica, dos5Dimensiones, SaltoY

- Dificultad: Aprendiz, Maestro

6. Dependencias Externas y Bibliotecas

Biblioteca / Modulo Qt	Uso en el proyecto
Qt Widgets	Ventana principal, QGraphicsView, QPushButton para botones de UI
Qt Multimedia	QMediaPlayer, QAudioOutput y QSoundEffect para musica de fondo y efectos
Qt Core	QTimer, QObject, senales/slots, QVector, QMap, QSet
Qt GUI	QPixmap, QPainter, QKeyEvent, QFont, QRectF
Qt GraphicsView	QGraphicsScene, QGraphicsObject, QGraphicsItem, QGraphicsPixmapItem
QRandomGenerator	Generacion de numeros aleatorios para IA y seleccion de sonidos

7. Desarrollo e Implementación

7.1 Sistema de animación por spritesheet

Cada personaje carga una hoja de sprites de 128x128 pixeles por frame, organizada en 32 filas (8 estados x 4 direcciones). El metodo seccionarSpritesheet() en Personaje recorre automaticamente la hoja y crea el mapa de animaciones indexado por la estructura AnimacionKey {Estado, Direccion, ZonaAtaque}.

7.2 Modos de física

La clase Bride implementa cuatro modos de física independientes activables segun contexto:

- Uniforme: movimiento lateral constante con velocidad fija para el nivel 1 (movRectilineo).
- SaltoY: salto vertical con gravedad aplicada frame a frame hasta colision con el suelo (movSaltoY).
- Parabólico: salto con componente horizontal (saltarDerecha/Izquierda), activable solo durante Zanshin especial (movParabolico).

- dos5Dimensiones: movimiento en ejes X e Y con aceleración, velocidad Maxima, fricción y desaceleración por tecla para el nivel 2 (movDosDimensiones).

7.3 Sistema Zanshin

Mecanica exclusiva del nivel 1. Al golpear a O-Ren con zonas de ataque distintas en secuencia (sin repetir la última zona usada), el contador Zanshin incrementa. Al llegar a 3 combinaciones distintas, se activa el Zanshin Especial durante 10 segundos: O-Ren entra en modo debilitado (ataques menos frecuentes, velocidad reducida a la mitad) y Bride desbloquea los saltos parabólicos y en el eje vertical.

7.4 IA adaptativa de Bill

Bill implementa un sistema de aprendizaje simplificado. Mantiene tres contadores por zona de ataque: ataques recibidos de Bride, parrys exitosos de Bride y golpes propios exitosos. Tras el mínimo de interacciones requerido (1 en Maestro, 2 en Aprendiz), recalcula probabilidades de ataque mediante pesos que penalizan las zonas más bloqueadas y refuerzan las más exitosas.

En modo Maestro: $\text{peso}[i] = (\text{ataques} * \text{exitosos}) / (\text{parrys} * \text{bloqueados})$.

En modo Aprendiz: $\text{peso}[i] = 1.0 + (\text{ataques} * 0.2)$. El resultado es un enemigo que aprende los patrones del jugador.

7.5 Detección de colisiones

Las hitboxes de ataque y cuerpo se calculan en coordenadas de escena mediante mapToScene(), evitando errores por transformaciones de escala del efecto 2.5D. La hitbox de ataque se desplaza según la dirección actual del personaje. Se usan banderas booleanas para evitar registrar múltiples impactos por el mismo ataque dentro del mismo frame.

7.6 Efecto de perspectiva 2.5D

En el nivel 2, la escala de cada personaje varia linealmente segun su posicion Y en la escena (entre ESCALA_MIN=1.2 y ESCALA_MAX=2.0 entre Y=200 y Y=550). Esto simula profundidad sin geometría 3D real. El cálculo se realiza en actualizarEscala25D() cada frame.

8. Procedimientos de Prueba

Las pruebas fueron principalmente manuales e iterativas .Se verificaron los siguientes aspectos:

8.1 Pruebas funcionales

- Transiciones correctas entre todos los estados de juego (menu -carga – nivel-victoria/derrota - menu).
- Registro de puntos solo al impactar hitboxes reales, sin doble conteo por frame.

- Activación y expiración del Zanshin Especial (10 segundos de duración, reinicio al recibir golpe).
- Parry exitoso vs. fallido según zona correcta y ventana de tiempo.
- Ajuste de probabilidades de Bill tras las interacciones mínimas requeridas.
- Detección de victoria y derrota en ambos niveles con emisión correcta de señales. Así mismo como la actualización de sprites.
- Para las colisiones se dibujaron los rectángulos temporalmente para evidenciar la posición de los hitboxes para cuerpo, zonas de ataque y el “arma”.

8.2 Verificaciones técnicas

- Carga correcta de recursos desde .qrc (verificado con qDebug() en constructores).
- Liberación de memoria al cambiar de nivel mediante qDeleteAll(niveles).
- Reproducción de audio sin superposición de pistas.
- Confinamiento de personajes a bordes definidos (verificarBordes).
- Ajuste de vista al cambiar de escena (fitInView en ajustarVista).

9. Controles del juego

Tecla (N1)	Accion (N1)	Tecla (N2)	Accion (N2)
A / D	Mover izq. / der.	A / D	Mover izq. / der.
J	Ataque Men (cabeza)	W / S	Mover atras / adelante
K	Ataque Do (torso)	Q / E	Acelerar / frenar
L	Ataque Kote (muneca)	J / K / L	Ataque Men / Do / Kote
Space	Saltar (Zanshin activo)	R + J/K/L	Parry Men / Do / Kote
Space+A/D	Salto parabólico		

10. Resultados y Discusion

10.1 Funcionamiento logrado

El sistema cumple todos los objetivos funcionales planteados: dos niveles con mecánicas diferenciadas, IA con comportamiento adaptativo, física de movimiento variada (cuatro modos), sistema de animación por spritesheet completo con 32 combinaciones de estado/dirección/ataque, audio integrado y gestión correcta del ciclo de vida del juego.

10.2 Limitaciones actuales

- La IA de O-Ren usa aleatoriedad simple sin aprendizaje; puede resultar predecible en partidas largas.
- No existe sistema de guardado de progreso ni puntuación persistente entre sesiones.
- Las hitboxes son rectangulares y no se ajustan perfectamente a la forma visual de los sprites.
- El juego carece de pantalla de opciones (control de volumen, cambio de resolución).

11. Conclusiones

El desarrollo de este videojuego permitió consolidar de manera práctica conceptos fundamentales de la programación orientada a objetos. La jerarquía Personaje -> Bride/Oren/Bill demuestra el valor de la herencia y el polimorfismo. El sistema de señales/slots de Qt materializa la conexión para loops. Y los diferentes modos de física en Bride ilustran como el principio de responsabilidad única puede convivir con la complejidad funcional.

La implementación de la IA adaptativa de Bill representa el logro técnico más ambicioso del proyecto, ya que introduce conceptos de aprendizaje automático simplificado en un contexto de videojuego en tiempo real, logrando un enemigo final que modifica su comportamiento según las acciones del jugador.

El proyecto concluye siendo un sistema funcional, extensible y con una arquitectura clara que puede servir de base para proyectos más complejos o como referencia didáctica para el estudio de POO aplicada al desarrollo de videojuegos con Qt C++.

12. Referencias

- [1] https://youtu.be/WNBuUtBA18U?si=1o-pl_SMLcEu2NPb. Acerca de enumeraciones para el uso en videojuegos.
- [2] <https://youtu.be/AIPYc71zuck?si=nUKXvFL0nu2FNjJJ>. Acerca de reproducción de audios y el uso QMediaPlayer.
- [3] <https://doc.qt.io/qt-6/qsoundeffect.html>. Acerca de QSoundeffect para sonidos de golpes.